



EEM 432

PROGRAMLANABİLİR MANTIK DENETLEYİCİLERİ

HOMEWORK

NAME-SURNAME:YAVUZ SELİM ÇAKMAK

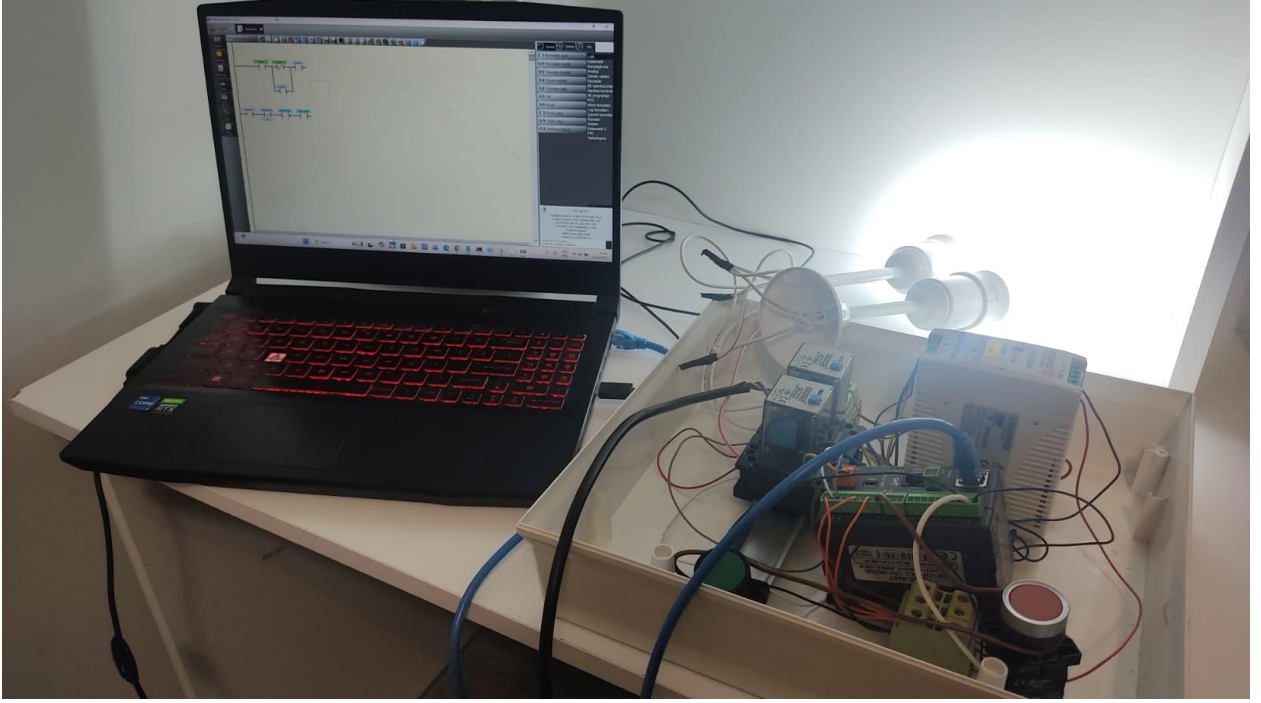
NUMBER:1201604007

NAME-SURNAME:ÖMERALP KOÇ

NUMBER:1211604067

PLC VE GÖRÜNTÜ İŞLEME ENTEGRE: Akıllı Hız Levhası Uyarı ve Yol Durum Tespit Sistemi

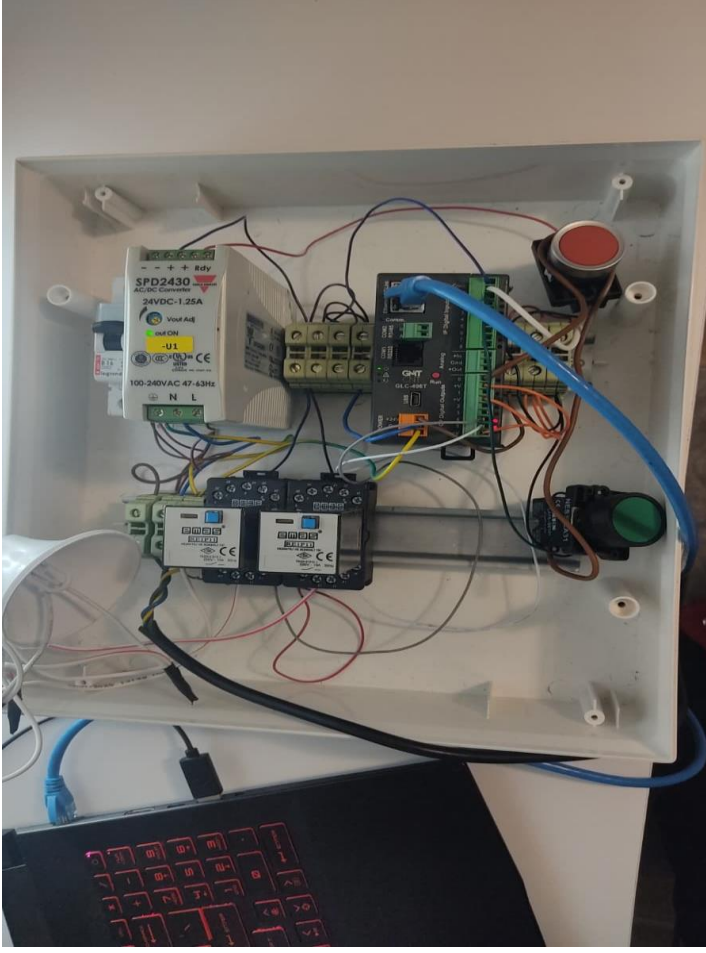
Bu rapor, Ömer Alp Koç ve Yavuz Selim Çakmak tarafından geliştirilen, görüntü işleme ve PLC (Programlanabilir Mantık Denetleyici) teknolojilerini entegre eden "**Akıllı Hız Levhası Uyarı ve Yol Durum Tespit Sistemi**" projesinin tüm teknik detaylarını, kurulum aşamalarını ve çalışma mantığını kapsamaktadır.



1. Proje Vizyonu ve Ana Mantık

Trafikte sürücülerin hız levhalarını gözden kaçırmaları ciddi riskler doğurabilir. Bu sistem, sürücüye yardımcı bir "asistan" görevi görür:

- **Şehir Dışı Tespiti (Hız > 55):** Sistem yol kenarındaki kırmızı halkalı hız levhasını okur. Eğer hız 55'in üzerindeyse (örneğin 80), araç içindeki ikaz lambası yanar. Sürücü bu sayede şehir içinde olmadığını ve hız sınırının arttığını anlar.
- **Şehir İçi Tespiti (Hız < 55):** Eğer okunan levha 55'in altındaysa (örneğin 50), ikaz lambası yanmaz. Sürücü bu durumda şehir içinde olduğunu algılar ve hızını 55 km/h limitinin altında tutması gerektiğini anlar. **Özetle; lamba yandığında şehir dışı, sönük olduğunda şehir içi trafiği algılanır.**



2. PLC Pano Devresi Elemanları

Sistem, endüstriyel standartlarda hazırlanmış bir pano üzerinde çalışmaktadır:

A. Güç ve Koruma Elemanları

- **SPD2430 AC/DC Konvertör (Güç Kaynağı):** Şebekeden gelen alternatif akımı, PLC ve butonlar için gerekli olan **24VDC 1.25A** doğru akıma çevirir.
- **Otomat Sigorta (C Tipi):** Sistemi kısa devre ve aşırı yüke karşı koruyan emniyet mekanizmasıdır.
- **Klemens Grupları:** Kablo karmaşasını önleyen ve güvenli elektrik dağıtımı sağlayan ray tipi bağlantı elemanlarıdır.

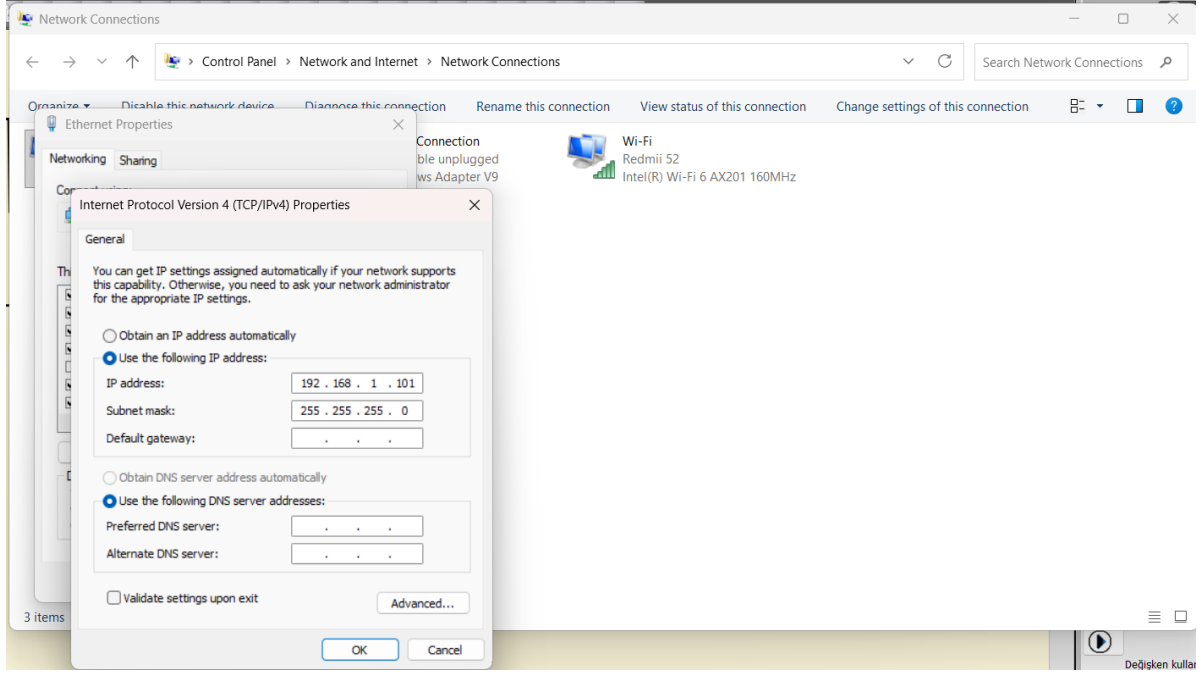
B. Kontrol ve Haberleşme Elemanları

- **GMT GLC-496T PLC:** Sistemin "beyni"dir; mühürleme ve karşılaştırma gibi tüm mantıksal işlemleri yürütür.
- **Mavi Ethernet Kablosu:** Python kodundan gelen verileri **Modbus TCP/IP** protokolü üzerinden PLC'ye aktarır.

C. Giriş ve Çıkış Elemanları

- **Yeşil Başlatma (Start) Butonu:** Sisteme enerji veren ve mühürlemeyi başlatan fiziksel giriştir (**IP 0**).
 - **Kırmızı Durdurma (Stop) Butonu:** Sistemi anında durduran fiziksel giriştir (**IP 1**).
 - **EMAS REIP11 Röleler:** PLC'nin transistör çıkışını kullanarak 220V ile çalışan lambayı anahtarlayan elemanlardır.
 - **Beyaz Lamba:** Hız sınırı aşıldığında yanan uyarıcıdır (**QP 0** çıkışına tanımlıdır).
-

3. Ağ Yapılandırması (IP Ayarları) ve Fiziksel Bağlantı



PLC ve bilgisayarın aynı kanal üzerinden haberleşmesi için IP yapılandırması şu şekilde yapılmıştır:

- **Ağ Kanalı Mantığı:** Her iki cihazın da IP adresindeki üçüncü hane (**1**) aynıdır (192.168.1.x). Bu, her iki cihazın aynı "kanalda" olduğunu gösterir.
- **Oda (IP) Ayırımı:** Çakışmayı önlemek için PLC'ye **100**, bilgisayara **101** "odası" (son hane) verilmiştir. PLC'nin IP adresi **192.168.1.100** olarak sabitlenmiştir.
- **PC Ayarı:** Denetim Masası -> Ağ ve Paylaşım Merkezi -> Ethernet Özellikleri -> IPv4 sekmesinden bilgisayarın IP adresi **192.168.1.101** olarak sabitlenmiştir.
- **Bağlantı Prosedürü:** PLC panosu prize takılıp sigorta açıldıktan sonra Ethernet kablosu bilgisayara bağlanır. PLC üzerindeki Ethernet ve turuncu lamba yandığında fiziksel bağlantı doğrulanmış olur.

4. Yazılım ve Kod Analizi

Sistemin görüntü işleme ayağı Python dilinde yazılmış ve terminal (**CMD**) üzerinden gerekli kütüphaneler yüklenmiştir:

- **Yüklü Kütüphaneler:** opencv-python, pytesseract, pymodbus ve numpy.

```

C:\Users\SASql>pip install opencv-python pymodbus pytesseract numpy
Collecting opencv-python
  Downloading opencv_python-4.12.0.88-cp37-abi3-win_amd64.whl (39.0 MB)
    39.0/39.0 MB 2.1 MB/s eta 0:00:00
Collecting pymodbus
  Downloading pymodbus-3.8.6-py3-none-any.whl (164 kB)
    164.4/164.4 kB 2.0 MB/s eta 0:00:00
Collecting pytesseract
  Downloading pytesseract-0.3.13-py3-none-any.whl (14 kB)
Requirement already satisfied: numpy in c:\users\sasql\anaconda3\lib\site-packages (1.21.5)
Collecting numpy
  Downloading numpy-2.0.2-cp39-cp39-win_amd64.whl (15.9 MB)
    15.9/15.9 MB 3.1 MB/s eta 0:00:00
Requirement already satisfied: Pillow>=8.0.0 in c:\users\sasql\anaconda3\lib\site-packages (from pytesseract) (9.2.0)
Requirement already satisfied: packaging>=21.3 in c:\users\sasql\anaconda3\lib\site-packages (from pytesseract) (21.3)
Requirement already satisfied: pyparsing!=3.0.5,>=2.0.2 in c:\users\sasql\anaconda3\lib\site-packages (from packaging>=21.3->pytesseract) (3.0.9)
Installing collected packages: pymodbus, numpy, pytesseract, opencv-python
  Attempting uninstall: numpy
    Found existing installation: numpy 1.21.5
    Uninstalling numpy-1.21.5:
      Successfully uninstalled numpy-1.21.5

```

- **Python kodu :**

```
import cv2
```

```
import pytesseract
```

```
import numpy as np
```

```
from pymodbus.client import ModbusTcpClient
```

```
import time
```

```
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'
```

```
PLC_IP = '192.168.1.100'
```

```
PLC_PORT = 502
```

```
REGISTER_ADRESI = 2000
```

```
client = ModbusTcpClient(PLC_IP, port=PLC_PORT)
```

```
client.connect()
```

```
def tabela_oku(frame):
```

```
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

```
    lower_red1 = np.array([0, 70, 50])
```

```
    upper_red1 = np.array([10, 255, 255])
```

```
    lower_red2 = np.array([170, 70, 50])
```

```

upper_red2 = np.array([180, 255, 255])

mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask = mask1 + mask2

contours, _ = cv2.findContours(mask, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

detected_speed = 0

for cnt in contours:
    area = cv2.contourArea(cnt)
    if area > 1000:
        perimeter = cv2.arcLength(cnt, True)
        approx = cv2.approxPolyDP(cnt, 0.04 * perimeter, True)

        if len(approx) > 5:
            x, y, w, h = cv2.boundingRect(cnt)
            roi = frame[y:y+h, x:x+w]
            gray_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
            _, thresh = cv2.threshold(gray_roi, 150, 255, cv2.THRESH_BINARY)

            text = pytesseract.image_to_string(thresh, config='--psm 11 -c
tessedit_char_whitelist=0123456789')

            text = text.strip()
            if text.isdigit():
                detected_speed = int(text)
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

```

```
        cv2.putText(frame, f"Hız: {detected_speed}", (x, y-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)
        return detected_speed
```

```
    return 0
```

```
cap = cv2.VideoCapture(0)
```

```
print("Sistem baslatildi")
```

```
try:
```

```
    while True:
```

```
        ret, frame = cap.read()
```

```
        if not ret:
```

```
            break
```

```
    hiz_degeri = tabela_oku(frame)
```

```
    if hiz_degeri > 0:
```

```
        print(f"Algılanan Hız: {hiz_degeri}")
```

```
    try:
```

```
        if not client.connected:
```

```
            print("Baglanti tazeleniyor...")
```

```
            client.connect()
```

```
        client.write_register(REGISTER_ADRESI, hiz_degeri)
```

```
    except Exception as e:
```

```

        print(f"PLC Haberlesme Hatasi: {e}")
        client.close()

cv2.imshow('Hiz Tabelasi Tespit', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

except KeyboardInterrupt:
    print("Program durduruluyor")

finally:
    cap.release()
    cv2.destroyAllWindows()
    client.close()

```

Python Kodunun Satır Analizi:

1. **Görüntü Alma:** cv2.VideoCapture(0) ile kameradan canlı yayın alınır.
2. **Renk Maskeleye:** HSV formatında kırmızı renk aralığı (lower_red ve upper_red) tanımlanarak levhanın çerçevesi bulunur.
3. **OCR (Metin Okuma):** pytesseract.image_to_string komutu, levha içindeki karakterleri okur ve int() fonksiyonu ile sayıya çevirir.
4. **Modbus Yazma:** Okunan değer, client.write_register(2000, hiz_degeri) komutu ile PLC'ye gönderilir.
5. **Haberleşme:** CMD üzerinden Python kodu açıldığında, ekranda "**Hız Tabelası Tespit**" adında kamerayı yansıtan bir pencere açılır. Terminalde "**PLC Bağlandı : 192.168.1.100**" yazısı görülür.


```
C:\Users\SASq1>python C:\Users\SASq1\OneDrive\Belgeler\Masaüstü\hiz_okumaa.py
PLC'ye bağlanıldı: 192.168.1.100
Algılanan Hız: 60
```

5. GMT Suite Konfigürasyonu ve Modbus Mantığı

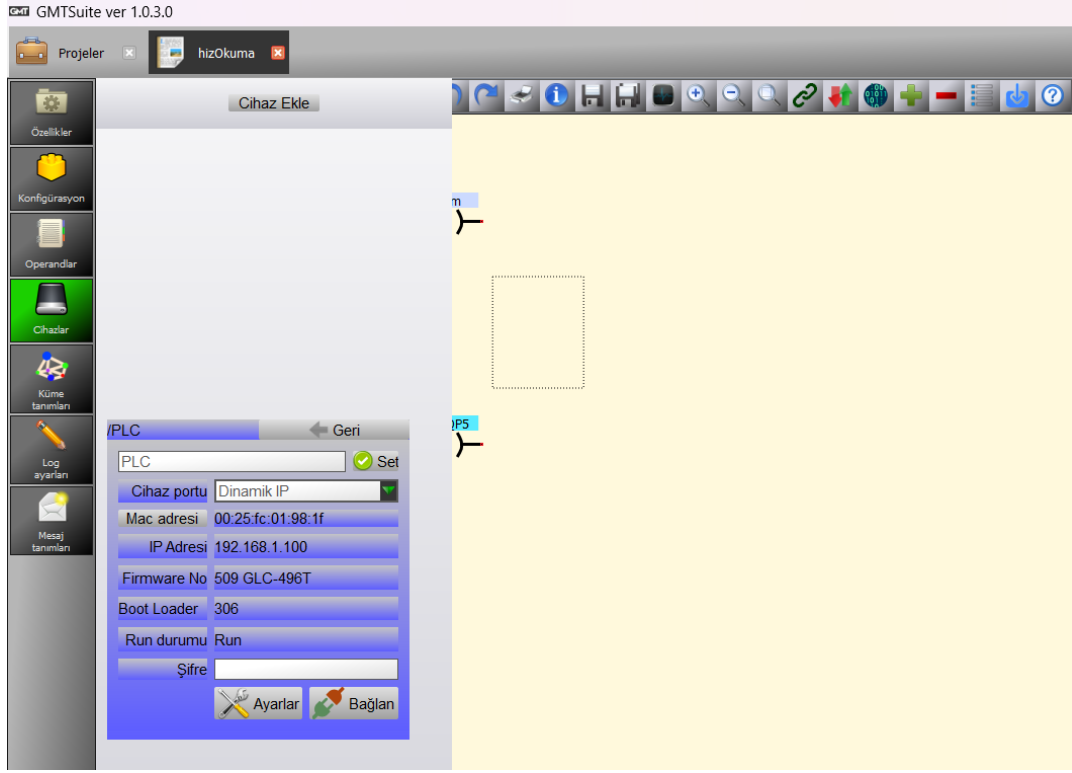
Haberleşmenin sorunsuz olması için şu adımlar izlenmiştir:

- **Cihaz Tanımlama:** "Cihazlar" sekmesinden PLC'ye bağlanılır (Şifre: **1234**). "Konfigürasyon" sekmesinden **GLC-496T** modeli seçilir.
- **Operand Tanımlama:** Start, stop ve sistem operatörleri tanımlanır. Lambaların bağlı olduğu **Q4 ve Q5** çıkışları konfigürasyonda isim değişikliği yapılmadan doğrudan atanmıştır.
- **MW2000 Operandı:** GMT PLC'de hız verisi için tanımlanan MW2000 operandı otomatik olarak **42001** adresine atanmıştır. Modbus sekmesindeki kutucuk işaretlenerek modbus haberleşme aktif hale getirilir.
- Register Hesaplaması: Python kütüphaneleri başlangıcı "0" kabul ettiği için aradaki fark "Offset" olarak koda yazılır:

$42001 \text{ (MW2000 Modbus Adresi)} - 40001 \text{ (Başlangıç Adresi)} = 2000 \text{ (Offset/Görelî Adres)}$

- **Veri Tipi:** Hız değerleri tam sayı olduğu için **Integer** seçilmiştir; bu sayede 0-300 arası değerler hassas şekilde taşınır.

No	Operand	Açıklama	Veri tipi	Açılış değeri	Kalıcı hafıza	ModBus	Modbus Adresi	Değerler	Kullanılma Adedi
6	CPU_IP6	IP 6	Bit		☐	☐			0
7	CPU_IP7	IP 7	Bit		☐	☐			0
8	CPU_IP8	IP 8	Bit		☐	☐			0
9	CPU_InpWord	Giriş word değeri	Word	0	☐	☐			0
85	CPU_LadderFreq	CPU Ladder tarama frekansı	Integer	0	☐	☐			0
86	CPU_LogStatus	CPU USB Log Durumu	Integer	0	☐	☐			0
88	CPU_MessageStatus	E-mail gönderim durumu	Integer	0	☐	☐			0
87	CPU_Message_Delay	Mesaj arası bekleme süresi	Integer	0	☐	☐			0
84	CPU_Osc_1000ms	1000 ms. Osilatör	Bit	0	☐	☐			0
82	CPU_Osc_100ms	100 ms. Osilatör	Bit	0	☐	☐			0
81	CPU_Osc_10ms	10 ms. Osilatör	Bit	0	☐	☐			0
80	CPU_Osc_1ms	1 ms. Osilatör	Bit	0	☐	☐			0
83	CPU_Osc_500ms	500 ms. Osilatör	Bit	0	☐	☐			0
16	CPU_OutWord	Çıkış word değeri	Word	0	☐	☐			0
10	CPU_QP0	QP 0	Bit	Off	☐	☐			0
11	CPU_QP1	QP 1	Bit	Off	☐	☐			0
12	CPU_QP2	QP 2	Bit	Off	☐	☐			0
13	CPU_QP3	QP 3	Bit	Off	☐	☐			0
14	CPU_QP4	QP 4	Bit	Off	☐	☐			1
15	CPU_QP5	QP 5	Bit	Off	☐	☐			1
59	CPU_RTC_Ay	RTC Ay 1..12	Integer	0	☐	☐			0
56	CPU_RTC_Dakika	RTC Dakika 0..59	Integer	0	☐	☐			0
58	CPU_RTC_Gun	RTC Gün 1..31	Integer	0	☐	☐			0
61	CPU_RTC_Haftagunindex	RTC Hafta gün no 1..7	Integer	0	☐	☐			0
55	CPU_RTC_Saat	RTC Saat 0..23	Integer	0	☐	☐			0
57	CPU_RTC_Saniye	RTC Saniye 0..59	Integer	0	☐	☐			0
60	CPU_RTC_Yil	RTC Yıl 15..99	Integer	0	☐	☐			0
75	CPU_SvDev0_timeout	Dev. 0 Timeout	Integer	1000	☐	☐			0
76	CPU_SvDev1_timeout	Dev. 1 Timeout	Integer	1000	☐	☐			0
77	CPU_SvDev2_timeout	Dev. 2 Timeout	Integer	1000	☐	☐			0
78	CPU_SvDev3_timeout	Dev. 3 Timeout	Integer	1000	☐	☐			0
79	CPU_SvDev4_timeout	Dev. 4 Timeout	Integer	1000	☐	☐			0
89	CPU_WMIStatus	WMI Server bağlantı durumu	Integer	0	☐	☐			0
90	MW2000	Hız verisi	Integer	0	☐	☒	42001		1
92	sistem		Bit	0	☐	☐			3
0	start	IP 0	Bit		☐	☐			1
1	stop	IP 1	Bit		☐	☐			1



6. Uygulama ve Sonuçlar

Fiziksel bağlantılar tamamlanıp (Ethernet ışıkları yandıktan sonra) şu adımlar izlenmiştir:

1. **GMT Suite:** GMT Suite üzerinden hazırlanan Ladder diyagramı şu mantıkla çalışır:
2. **Mühürleme (Latch):** Start butonuna (IP 0) basıldığında sistem biti aktif olur. El çekilse dahi sistem kontağı üzerinden elektrik akmaya devam eder, böylece sistem sürekli aktif kalır.
3. **Karşılaştırma Operatörü:** Sistem aktifken, Python'dan gelen MW2000 değeri sürekli **> 55** (büyüktür 55) şartına tabi tutulur.
4. **Çıkış Tetikleme:**
5. **70 km/h Durumu:** Karşılaştırma "Doğru" döner. QP4 ve QP5 (veya QP0) çıkışları 1 olur, pano üzerindeki lamba yanar. Terminalde "Algılanan Hız: 70" yazısı akar.

6. **50 km/h Durumu:** Karşılaştırma "Yanlış" döner. Çıkışlar 0 olur, lamba söner. Terminalde "Algılanan Hız: 50" yazısı görülür.

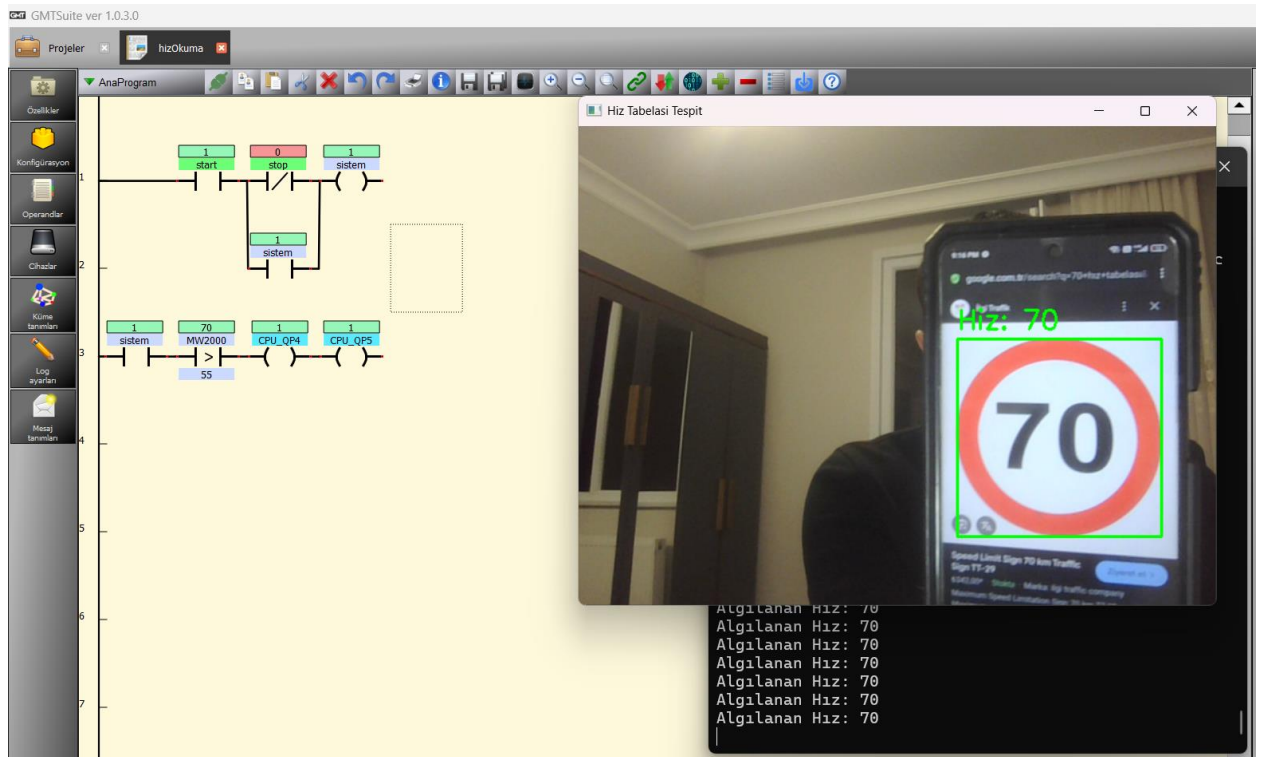
Programın Devreye Alınması:

1. GMT Suite sol üstteki "**Cihaz**" (yeşil fiş/kırmızı priz) butonuna tıklanır.
2. "**Program Yükle**" denilerek diyagram PLC'ye atılır.
3. "**PLC Program Doğrula**" ve ardından "**PLC Program Çalış**" komutları verilir.
4. Zincir amblemi olan "**Çevrimiçi Bağlantı**" butonuna tıklanarak sistem canlı takibe alınır.

Çalıştırma: CMD üzerinden Python kodu başlatıldı. Ekranda "**Hız Tabelası Tespit**" penceresi açıldı ve terminalde "**PLC Bağlandı**" yazısı görüldü.

Başarı Kanıtı:

- **70 km/h Testi:** Kameraya 70 levhası gösterildiğinde, Python saniyede 10 kez "Algılanan hız: 70" yazdı. PLC diyagramında MW2000 hücreinde 70 değeri görüldü, karşılaştırma aktif oldu ve **QP4/QP5** üzerinden lamba anında yandı.



- **50 km/h Testi:** Levha değıştiğinde sistem 50'yi okudu. Değer 55'ten küçük olduğu için çıkışlar sıfırlandı ve lamba söndü.

GMTSuite ver 1.0.3.0

Projeler x hızOkuma x

AnaProgram

1 start 0 stop 1 sistem

2 sistem

3 sistem MW2000 50 CPU_QP4 0 CPU_QP5 0

4

5

6

7

Hız Tabelası Tespit

4:11 PM

google.com.tr: 30x30/70x50-hiz-tabelasi...

Hız: 50

50

Azami Hız Sınırlaması Levhası
80 km Fiyatı Yıl Azami Hız...

6342,00 - Hızla - Marka: 100 Trafik

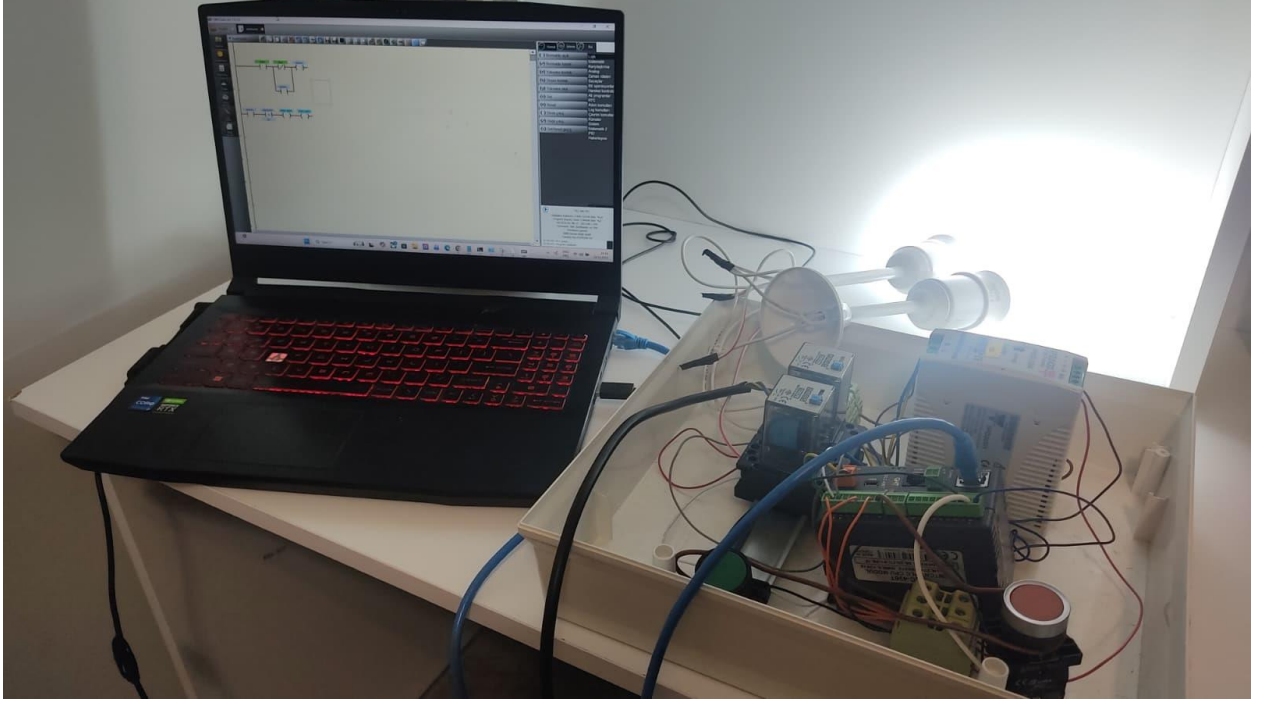
Azami Hız Sınırlaması Levhası 80 km T1 (20...
Azami Hız Sınırlaması Levhası 80 km T1 (20...
Fisane Levhası ve diğer durumları öğrenmek için web sitesine
görmek için. Görmek için web sitesine gidebilirsiniz. Daha Fazla Bilgi

< Geri

Kayıt

Algılanan Hız: 50
Algılanan Hız: 50
Algılanan Hız: 50
Algılanan Hız: 50
Algılanan Hız: 50
Algılanan Hız: 50
Algılanan Hız: 50

- **Genel Sonuç:** Sistem 30, 40, 80 ve 100 gibi tüm hız levhası tiplerinde başarıyla test edilmiştir.



Gelecek Potansiyeli

Bu sistem ileride araçların gaz pedalına entegre edilerek **otomatik hız sabitleyici** haline getirilebilir veya sanayi tesislerinde konveyör hatlarının hızını levhalarla dinamik olarak yönetmek için kullanılabilir.